

C++编程基础1

1. C++函数

1.1 返回类型

1.2 函数名

1.3 函数的形参列表

1.4 函数体

1.5 输入输出流

1.6 举例

2. 基础

2.1 C++输入输出数据

2.2 程序的注释

2.3 使用变量

2.4 使用常量

2.5 标识符的命名

3. 运算

3.1 算术运算

3.2 自增和自减

3.3 赋值运算

3.4 C++11初始化赋值

3.5 关系运算

3.6 逻辑运算

3.7 运算的优先级

4. 条件结构

4.1 if语句的基本结构

4.2 嵌套使用if语句

4.3 多条件的if语句

4.4 if语句中的逻辑表达式

4.5 三目运算

4.6 switch语句

5. 循环结构

5.1 while循环语句

5.2 循环的跳转

5.3 for循环语句

5.4 嵌套使用循环

5.5 do...while循环

5.6 goto语句

6. 作业

1. C++函数

程序的启动都是从main函数开始的

```
▼ main C++ |
1 ▼ int main() {
2     return 0;
3 }
```

该函数包括了返回类型、函数名称、形参列表、函数体四个部分

1.1 返回类型

int float void double

返回一个引用？

但是main的返回类型只能是int

1.2 函数名

大小驼峰/下划线命名法

1.3 函数的形参列表

表示传入的参数，常常会传入引用（引用传递/值传递）

1.4 函数体

即该函数的实现，return用于返回特定的值

1.5 输入输出流

iostream

这个库中常用的两个对象是cin && cout

1.6 举例

```
▼ C++函数举例 C++ |
1  #include <iostream>
2
3  // 函数声明：传递一个常量引用，并返回一个常量引用
4  const int& findMax(const int& a, const int& b);
5
6  int main() {
7      int x = 10;
8      int y = 20;
9
10     // 调用findMax函数并输出结果
11     const int& max = findMax(x, y);
12     std::cout << "The maximum value is: " << max << std::endl;
13
14     return 0;
15 }
16
17 // 函数定义：使用const引用传递参数并返回const引用
18 const int& findMax(const int& a, const int& b) {
19     return (a > b) ? a : b;
20 }
21
```

2. 基础

2.1 C++输入输出数据

在C++中，常用的输入输出流分别是 `cin` 和 `cout` 。它们属于标准输入输出库，即 `iostream`

```

1  #include <iostream> // 包含输入输出流库
2  int main() {
3      int number;
4      std::cout << "请输入一个数字: "; // 输出提示信息
5      std::cin >> number; // 从标准输入读取数据
6      std::cout << "你输入的数字是: " << number << std::endl; // 输出数据
7      return 0;
8  }

```

在这个例子中，`std::cout` 用于输出，`std::cin` 用于输入，`std::endl` 用于换行。

2.2 程序的注释

C++支持两种注释方式：

单行注释：用 `//` 表示。

多行注释：用 `/* */` 表示。

```

1  #include <iostream>
2  int main() {
3      // 这是一个单行注释
4      /*
5         这是一个
6         多行注释
7      */
8      std::cout << "Hello, World!" << std::endl;
9      return 0;
10 }

```

2.3 使用变量

变量是存储数据的基本单元。在C++中，变量必须先声明后使用。常见的变量类型有 `int`、`float`、`double`、`char`、`bool` 等。

```

1  #include <iostream>
2  int main() {
3      int age = 25; // 整型变量
4      float height = 1.75; // 浮点型变量
5      double weight = 70.5; // 双精度浮点型变量
6      char gender = 'M'; // 字符型变量
7      bool isStudent = true; // 布尔型变量
8      std::cout << "Age: " << age << std::endl;
9      std::cout << "Height: " << height << std::endl;
10     std::cout << "Weight: " << weight << std::endl;
11     std::cout << "Gender: " << gender << std::endl;
12     std::cout << "Is Student: " << std::boolalpha << isStudent << std::endl;
13 }
14 return 0;
15 }
```

2.4 使用常量

常量是不可改变的值。在C++中，可以使用 `const` 关键字定义常量。

```

1  #include <iostream>
2  int main() {
3      const double PI = 3.14159; // 定义常量
4      std::cout << "The value of PI is: " << PI << std::endl;
5      // PI = 3.14; // 错误，常量的值不能被修改
6      return 0;
7  }
```

2.5 标识符的命名

标识符是变量、函数、类等名字。在C++中，标识符命名需遵循以下规则：

1. 只能使用字母（a-z，A-Z）、数字（0-9）和下划线（_）。
2. 不能以数字开头。
3. 区分大小写。
4. 不能使用C++关键字（如 `int`、`return` 等）。

良好的命名规范：

- 变量名和函数名通常使用小写字母，多个单词可以用下划线或驼峰命名法，如 `student_name` 或 `studentName`。
- 常量名通常使用全大写字母和下划线，如 `MAX_LENGTH`。

```
1  #include <iostream>
2  int main() {
3      int studentAge = 20;
4      const int MAX_AGE = 100;
5      std::cout << "Student Age: " << studentAge << std::endl;
6      std::cout << "Max Age: " << MAX_AGE << std::endl;
7      return 0;
8  }
```

3. 运算

3.1 算术运算

C++中常见的算术运算符包括加、减、乘、除和取模。它们分别是 `+`、`-`、`*`、`/` 和 `%`。

```
1  #include <iostream>
2  int main() {
3      int a = 10;
4      int b = 3;
5      std::cout << "a + b = " << a + b << std::endl;
6      std::cout << "a - b = " << a - b << std::endl;
7      std::cout << "a * b = " << a * b << std::endl;
8      std::cout << "a / b = " << a / b << std::endl;
9      std::cout << "a % b = " << a % b << std::endl;
10     return 0;
11 }
```

3.2 自增和自减

自增运算符 `++` 和自减运算符 `--` 用于对变量进行加1或减1操作。它们有前置和后置两种形式：

```

1  #include <iostream>
2  int main() {
3      int x = 5;
4      std::cout << "x++ = " << x++ << std::endl; // 输出5, x变为6
5      std::cout << "++x = " << ++x << std::endl; // 输出7
6      std::cout << "x-- = " << x-- << std::endl; // 输出7, x变为6
7      std::cout << "--x = " << --x << std::endl; // 输出5
8      return 0;
9  }

```

注意前置和后置的区别!

3.3 赋值运算

赋值运算符 `=` 用于将右侧表达式的值赋给左侧变量。复合赋值运算符包括 `+=`、`-=`、`*=`、`/=` 和 `%=` 等。

```

1  #include <iostream>
2  int main() {
3      int a = 10;
4      int b = 3;
5      a += b; // 相当于 a = a + b
6      std::cout << "a += b: " << a << std::endl;
7      a -= b; // 相当于 a = a - b
8      std::cout << "a -= b: " << a << std::endl;
9      a *= b; // 相当于 a = a * b
10     std::cout << "a *= b: " << a << std::endl;
11     a /= b; // 相当于 a = a / b
12     std::cout << "a /= b: " << a << std::endl;
13     a %= b; // 相当于 a = a % b
14     std::cout << "a %= b: " << a << std::endl;
15     return 0;
16 }

```

3.4 C++11初始化赋值

C++11引入了新的初始化语法，包括列表初始化和统一初始化方式。

```

1  #include <iostream>
2  #include <vector>
3  int main() {
4      int x = 5; // 传统初始化
5      int y{10}; // 列表初始化
6      int z = {15}; // 统一初始化
7      std::vector<int> vec{1, 2, 3, 4, 5}; // 列表初始化用于容器
8      std::cout << "x: " << x << ", y: " << y << ", z: " << z << std::endl;
9      std::cout << "vec: ";
10     for (int n : vec) {
11         std::cout << n << " ";
12     }
13     std::cout << std::endl;
14     return 0;
15 }

```

3.5 关系运算

关系运算符用于比较两个值，结果为布尔值 `true` 或 `false`。常见的关系运算符有 `==`、`!=`、`>`、`<`、`>=` 和 `<=`。

```

1  #include <iostream>
2  int main() {
3      int a = 5;
4      int b = 3;
5      std::cout << "a == b: " << (a == b) << std::endl;
6      std::cout << "a != b: " << (a != b) << std::endl;
7      std::cout << "a > b: " << (a > b) << std::endl;
8      std::cout << "a < b: " << (a < b) << std::endl;
9      std::cout << "a >= b: " << (a >= b) << std::endl;
10     std::cout << "a <= b: " << (a <= b) << std::endl;
11     return 0;
12 }

```

3.6 逻辑运算

逻辑运算符用于逻辑运算，包括逻辑与 `&&`、逻辑或 `||` 和逻辑非 `!`。

```

1  #include <iostream>
2  int main() {
3      bool x = true;
4      bool y = false;
5      std::cout << "x && y: " << (x && y) << std::endl;
6      std::cout << "x || y: " << (x || y) << std::endl;
7      std::cout << "!x: " << (!x) << std::endl;
8      return 0;
9  }

```

3.7 运算的优先级

C++中运算符有不同的优先级。优先级高的运算符先计算，优先级相同则按结合性规则计算。常见的优先级顺序如下：

1. 括号 `()` 最高优先级
2. 自增自减 `++ --`
3. 算术运算 `*` `/` `%` 高于 `+` `-`
4. 关系运算 `<` `<=` `>` `>=`
5. 相等运算 `==` `!=`
6. 逻辑与 `&&` 高于逻辑或 `||`
7. 赋值运算 `=` `+=` `-=` `*=` `/=` `%=`

```

1  #include <iostream>
2  int main() {
3      int a = 5, b = 3, c = 2;
4      int result = a + b * c; // 计算结果为11, 因为乘法优先于加法
5      std::cout << "Result: " << result << std::endl;
6      return 0;
7  }

```

4. 条件结构

4.1 if语句的基本结构

`if` 语句用于根据条件的真假来执行代码块。基本结构如下：

```
1  #include <iostream>
2  int main() {
3      int x = 10;
4      if (x > 5) {
5          std::cout << "x is greater than 5" << std::endl;
6      }
7      return 0;
8  }
```

如果 `x > 5` 为真，代码块中的语句将被执行。

4.2 嵌套使用if语句

可以在一个 `if` 语句的代码块中嵌套另一个 `if` 语句，用于处理更复杂的条件判断。

```
1  #include <iostream>
2  int main() {
3      int x = 10;
4      if (x > 5) {
5          if (x < 15) {
6              std::cout << "x is between 5 and 15" << std::endl;
7          }
8      }
9      return 0;
10 }
```

4.3 多条件的if语句

多条件的 `if` 语句可以使用 `else if` 和 `else` 来处理多个条件。

```

1  #include <iostream>
2  int main() {
3      int x = 10;
4      if (x > 15) {
5          std::cout << "x is greater than 15" << std::endl;
6      } else if (x > 5) {
7          std::cout << "x is between 6 and 15" << std::endl;
8      } else {
9          std::cout << "x is 5 or less" << std::endl;
10     }
11     return 0;
12 }

```

4.4 if语句中的逻辑表达式

if 语句中的条件可以是逻辑表达式，例如逻辑与 `&&`、逻辑或 `||` 和逻辑非 `!`。

```

1  #include <iostream>
2  int main() {
3      int x = 10;
4      int y = 20;
5      if (x > 5 && y < 25) {
6          std::cout << "x is greater than 5 and y is less than 25" << std::e
7          ndl;
8      }
9      if (!(x > 15)) {
10         std::cout << "x is not greater than 15" << std::endl;
11     }
12     return 0;
13 }

```

4.5 三目运算

三目运算符（条件运算符）是一种简写的 if-else 语句。其语法如下： `condition ? expression1 : expression2`。如果条件为真，返回 `expression1`，否则返回 `expression2`。

```

1  #include <iostream>
2  int main() {
3      int x = 10;
4      int y = (x > 5) ? 100 : 200; // 如果x大于5, 则y为100, 否则y为200
5      std::cout << "y: " << y << std::endl;
6      return 0;
7  }

```

4.6 switch语句

`switch` 语句用于根据变量的值执行不同的代码块。它常用于替代多个 `if-else` 语句。每个 `case` 表示一个可能的值, `break` 语句用于终止当前 `case`。

```

1  #include <iostream>
2  int main() {
3      int x = 2;
4      switch (x) {
5          case 1:
6              std::cout << "x is 1" << std::endl;
7              break;
8          case 2:
9              std::cout << "x is 2" << std::endl;
10             break;
11          case 3:
12              std::cout << "x is 3" << std::endl;
13              break;
14          default:
15              std::cout << "x is not 1, 2, or 3" << std::endl;
16              break;
17      }
18      return 0;
19  }

```

5. 循环结构

5.1 while循环语句

`while` 循环用于在条件为真时重复执行一段代码。基本结构如下:

```

1  #include <iostream>
2  int main() {
3      int x = 0;
4      while (x < 5) {
5          std::cout << "x: " << x << std::endl;
6          x++;
7      }
8      return 0;
9  }

```

在上面的例子中，只要 `x` 小于5，循环将继续执行。

5.2 循环的跳转

在循环中可以使用 `break` 和 `continue` 语句来跳转。

- `break` 语句用于终止循环。
- `continue` 语句用于跳过当前循环的剩余部分，并进入下一次循环。

```

1  #include <iostream>
2  int main() {
3      int x = 0;
4      while (x < 10) {
5          if (x == 5) {
6              break; // 终止循环
7          }
8          std::cout << "x: " << x << std::endl;
9          x++;
10     }
11     for (int y = 0; y < 10; y++) {
12         if (y % 2 == 0) {
13             continue; // 跳过偶数
14         }
15         std::cout << "y: " << y << std::endl;
16     }
17     return 0;
18 }

```

5.3 for循环语句

`for` 循环用于在已知循环次数的情况下重复执行代码。基本结构如下：

```
1 #include <iostream>
2 int main() {
3     for (int i = 0; i < 5; i++) {
4         std::cout << "i: " << i << std::endl;
5     }
6     return 0;
7 }
```

在上面的例子中，`i` 从0开始，每次循环 `i` 增加1，直到 `i` 小于5。

5.4 嵌套使用循环

可以在一个循环内部嵌套另一个循环，用于处理多维数据结构等复杂情况。

```
1 #include <iostream>
2 int main() {
3     for (int i = 0; i < 3; i++) {
4         for (int j = 0; j < 3; j++) {
5             std::cout << "i: " << i << ", j: " << j << std::endl;
6         }
7     }
8     return 0;
9 }
```

在上面的例子中，外层循环执行3次，每次外层循环执行时，内层循环也执行3次。

5.5 do...while循环

`do...while` 循环与 `while` 循环类似，但 `do...while` 循环至少执行一次，因为条件是在循环体之后检查的。

```
1  #include <iostream>
2  int main() {
3      int x = 0;
4      do {
5          std::cout << "x: " << x << std::endl;
6          x++;
7      } while (x < 5);
8      return 0;
9  }
```

在上面的例子中，无论条件是否为真，循环体至少执行一次。

5.6 goto语句

`goto` 语句用于无条件跳转到程序中的某个标签位置。一般不推荐使用，因为它会使代码难以阅读和维护。

```
1  #include <iostream>
2  int main() {
3      int x = 0;
4      start:
5      std::cout << "x: " << x << std::endl;
6      x++;
7      if (x < 5) {
8          goto start; // 跳转到start标签
9      }
10     return 0;
11 }
```

6. 作业

- 服务器每次登录都需要输入密码，尝试解决这个问题
- 用C++和python解决下面这几个问题

题目描述

你的任务是计算 $a+b$ 。

输入描述

输入包含一系列的 a 和 b 对，通过空格隔开。一对 a 和 b 占一行。

输出描述

对于输入的每对 a 和 b ，你需要依次输出 a 、 b 的和。
如对于输入中的第二对 a 和 b ，在输出中它们的和应该也在第二行。

输入示例

```
3 4
11 40
```

输出示例

```
7
51
```

题目描述

计算 $a+b$ ，但输入方式有所改变。

输入描述

第一行是一个整数 N ，表示后面会有 N 行 a 和 b ，通过空格隔开。

输出描述

对于输入的每对 a 和 b ，你需要在相应的行输出 a 、 b 的和。
如第二对 a 和 b ，对应的和也输出在第二行。

输入示例

```
2
2 4
9 21
```

输出示例

```
6
30
```

提示信息

注意，测试数据不仅仅一组。也就是说，会持续输入 N 以及后面的 a 和 b

题目描述

你的任务依然是计算 $a+b$ 。

输入描述

输入中每行是一对 a 和 b 。其中会有一对是0和0标志着输入结束，且这一对不要计算。

输出描述

对于输入的每对 a 和 b ，你需要在相应的行输出 a 、 b 的和。
如第二对 a 和 b ，他们的和也输出在第二行。

输入示例

```
2 4
11 19
0 0
```

输出示例

```
6
30
```

题目描述

你的任务是计算若干整数的和。

输入描述

每行的第一个数 N ，表示本行后面有 N 个数。
如果 $N=0$ 时，表示输入结束，且这一行不要计算。

输出描述

对于每一行数据需要在相应的行输出和。

输入示例

```
4 1 2 3 4
5 1 2 3 4 5
0
```

输出示例

```
10
15
```

题目描述

你的任务是计算若干整数的和。

输入描述

输入的第一行为一个整数N，接下来N行每行先输入一个整数M，然后在同一行内输入M个整数。

输出描述

对于每组输入，输出M个数的和，每组输出之间输出一个空行。

输入示例

```
3
4 1 2 3 4
5 1 2 3 4 5
3 1 2 3
```

输出示例

```
10
15
6
```

提示：使用chatGPT来辅助你的工作，注意做好目录管理

- 拉取远程的仓库：git clone git@github.com:velonSigs/J.LEARN.git，本地创建一个自己的分支，并把这个分支推到远程仓库上去并提交PR与dev合并